

PROJECT REPORT

Sentra

AI-Powered Sentiment Analysis Web Application

Machine learning | Python API | Firebase | Vercel

LIVE APP
sentra-ai-powered-sentiment-analysi.vercel.app

GITHUB
github.com/scientistayat-engineer/Sentra-AI-Powered-Sentiment-Analysis

SUBMITTED BY
Anushay Ayat
Artificial Intelligence and Data Science
Saylani Mass IT Training (SMIT)

October 2026

Contents

1 Abstract
2 Introduction and objectives
3 Technology stack
4 System architecture
5 Dataset
6 Text preprocessing
7 Feature extraction and model training
8 Evaluation and results
9 Backend API design
10 Frontend design
11 Firebase integration and security
12 Deployment
13 Testing
14 Challenges and limitations
15 Future work
16 Conclusion
17 References
A Appendix: running the project

1. Abstract

Sentra is a full-stack web application that reads a piece of English text, such as a customer review or comment, and classifies its sentiment as **Positive**, **Negative** or **Neutral**. The prediction comes from a real machine-learning model: text is cleaned, converted to TF-IDF features and classified by a scikit-learn model that was selected on a validation set and scored once on a held-out test set. A Flask API serves the model, a vanilla HTML, CSS and JavaScript frontend presents the results, and Firebase provides user accounts and a private prediction history. The application is deployed on Vercel and the source code is hosted on GitHub.

On the 12,284 held-out test texts the selected model reaches an accuracy of **58.8%** and a macro F1-score of **58.4%**. This report documents the problem, the technology choices, the machine-learning workflow, the API and frontend design, the Firebase security model, deployment, testing and the limitations of the system.

Project link	Address
Live application	https://sentra-ai-powered-sentiment-analysi.vercel.app
Source code (GitHub)	https://github.com/scientistayat-engineer/Sentra-AI-Powered-Sentiment-Analysis

2. Introduction and objectives

Organisations receive large volumes of written feedback through reviews, social media and support channels. Reading all of it manually is slow and subjective. Sentiment analysis assigns each text a consistent label, which lets a team see the overall mood and find unhappy customers quickly.

The project objectives were to:

- apply machine learning and natural language processing to a real text-classification task;
- build a modern, responsive and accessible frontend using only HTML, CSS and JavaScript;
- expose the model through a validated Python REST API that returns structured JSON;
- integrate Firebase Authentication and Cloud Firestore in a meaningful and secure way;
- deploy the application publicly on Vercel and manage the code on GitHub;
- report genuine evaluation results rather than hard-coded or random predictions.

3. Technology stack

Layer	Technology	Purpose
Frontend	HTML5, CSS3, JavaScript (ES modules)	Pages, styling, animation, API calls, charts
Backend	Python 3.12, Flask	REST API, validation, error handling

Layer	Technology	Purpose
Machine learning	scikit-learn, pandas, NumPy, joblib	Features, training, evaluation, model storage
Accounts and data	Firebase Authentication, Cloud Firestore	Sign-in and private prediction history
Hosting	Vercel	Static frontend and Python serverless function
Version control	Git and GitHub	Source code and documentation

No frontend framework is used, in line with the assignment requirements. Flask was chosen for the backend because it is small, well documented and runs on Vercel as a serverless WSGI function.

4. System architecture

The system has three cooperating parts. The **browser frontend** consists of three static pages (home with analyzer, login, dashboard) whose JavaScript modules call the API with `fetch` and talk to Firebase through its web SDK. The **Python API** is a Flask application with three endpoints; it loads the trained model once per process and keeps it in memory, which keeps responses fast on serverless platforms. **Firebase** authenticates the user and stores each saved analysis under that user's own document path.

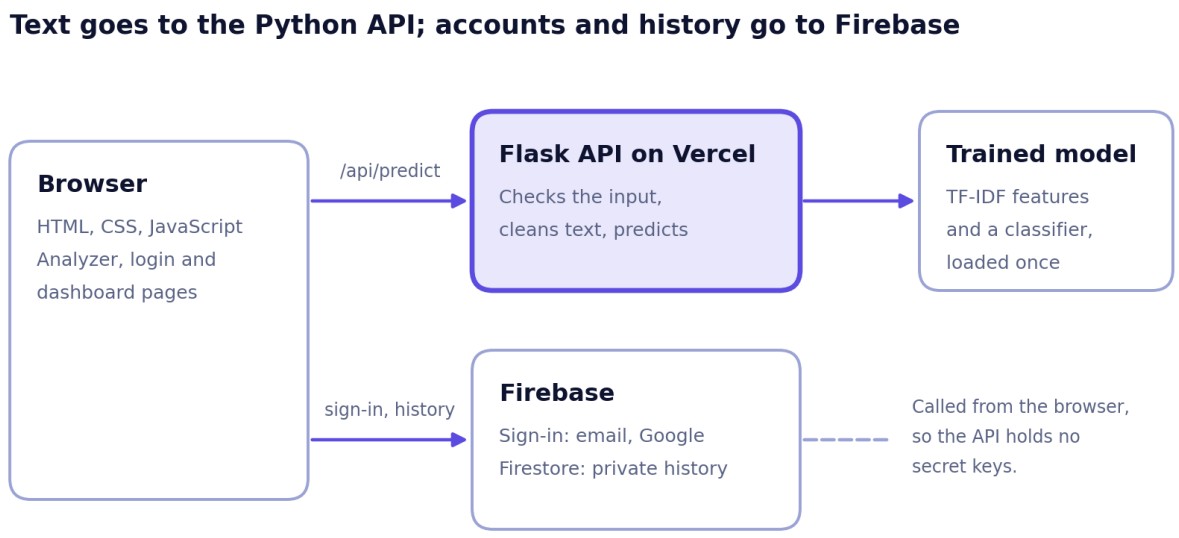


Figure 1. System architecture: the browser calls the API and Firebase

The browser sends text to `/api/predict` and receives a prediction. If the user is signed in, the browser then writes the result to Firestore directly. Because the Firebase web SDK is used from the browser, the Python backend never needs a service-account key, and no private credential exists in the repository. On Vercel the static files and the API share one domain, so cross-origin problems do not arise; the API still supports an `ALLOWED_ORIGINS` setting for deployments where it is hosted elsewhere.

Project structure

- `frontend/` holds the pages, `css/styles.css` and the JavaScript modules (`api.js`, `firebase.js`, `common.js`, `analyzer.js`, `auth.js`, `dashboard.js`).
- `backend/app/` holds `main.py` (API), `predictor.py` (model loading and prediction), `preprocess.py` (text cleaning) and `config.py`; `backend/tests/` holds the automated tests.
- `api/index.py` is the Vercel entry point; `vercel.json` configures routing and packaging.
- `ml/train.py` performs data loading, model selection, evaluation and export; results go to `ml/artifacts/` and the model to `models/`.
- `firebase/` holds the Firestore security rules and the setup guide.

5. Dataset

The model is trained on **TweetEval: Sentiment**, the SemEval-2017 Task 4 (Subtask A) data distributed by the Cardiff NLP group on Hugging Face (`cardiffnlp/tweet_eval`, configuration `sentiment`). Each text carries one of three labels: negative, neutral or positive.

Split	Texts	Used for
Training	45,615	Fitting the vectorizer and the models
Validation	2,000	Choosing the best candidate
Test	12,284	One final, unbiased evaluation

This dataset was chosen because it has exactly the three classes the assignment requires, including a genuine Neutral class; its splits are fixed, so results can be compared with published work; and it is openly available and well documented. The training script can also read any CSV with `text` and `label` columns (or 1 to 5 star ratings with the `--stars` flag), which makes it possible to retrain on product-review data without changing code.

6. Text preprocessing

One function, `clean_text`, is used both in training and at prediction time, so the model always sees text prepared in the same way. It performs these steps:

1. Unicode normalisation and conversion to lower case.
2. Web links are replaced by the token `url` and @mentions by the token `user`.
3. Hashtags keep their word (`#great` becomes `great`).
4. Letters repeated three or more times are shortened (`soooo` becomes `soo`).
5. Characters other than letters, digits, apostrophes, `!` and `?` become spaces, and whitespace is collapsed.

Stop words are deliberately kept

Words such as not, no and never flip the meaning of a sentence, so removing them would destroy the signal the model needs. Keeping ! and ? preserves emphasis, which also carries sentiment.

7. Feature extraction and model training

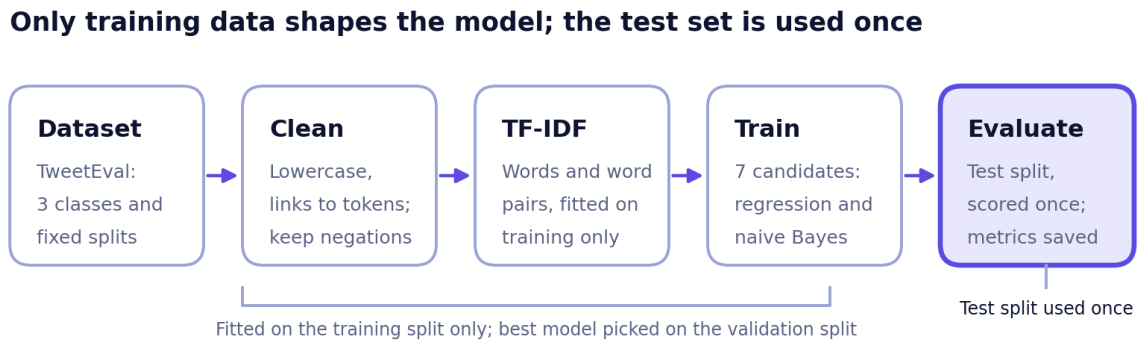


Figure 2. Training pipeline: each stage sees only the data it is allowed to

7.1 Features

Cleaned text is converted to numbers with a **TF-IDF vectorizer** using unigrams and bigrams (single words and word pairs), sub-linear term frequency, a minimum document frequency of two and at most 200,000 features. Bigrams let the model capture short phrases such as *not good*. The vectorizer is fitted on the training split only, which prevents information from the validation and test data leaking into the model.

7.2 Candidate models

Seven candidates are trained and compared: Logistic Regression with regularisation strength C of 0.5, 1, 2, 4 and 8 (balanced class weights, so the smaller classes are not ignored), and Complement Naive Bayes with smoothing of 0.3 and 1.0, a fast baseline well suited to text. The candidate with the highest **macro F1 on the validation split** is selected. The test split is not used for any decision. Both model families expose class probabilities, which the application uses to display confidence.

7.3 Explainability

For each prediction the API also returns the words that influenced the result. For every word in the model vocabulary, the model's per-class weight is multiplied by the word's TF-IDF value, and the class it leans towards is reported. Common function words are excluded from the highlighting (but not from the model). The frontend highlights these words in the user's text, which makes each verdict easier to trust.

8. Evaluation and results

8.1 Method

The selected model is evaluated **once** on the untouched test split. Precision, recall and F1 are averaged across the three classes (macro average), so the harder Neutral class counts as much as the others:

- **Accuracy:** the share of texts classified correctly.
- **Precision:** of the texts predicted as a class, the share that truly belong to it.
- **Recall:** of the texts that truly belong to a class, the share that was found.
- **F1-score:** the harmonic mean of precision and recall.

The training script writes `ml/artifacts/metrics.json`, `classification_report.txt`, `model_comparison.csv` and `confusion_matrix.png`. The model file also stores these metrics, and the home page reads them live from `/api/model-info`, so the numbers shown to users always match the deployed model.

8.2 Results

> **Test set: accuracy 58.8%, macro F1 58.4%**
>
> Selected model: Logistic Regression (C = 4), scored once on 12,284 texts it never saw during training.

Metric (test set)	Macro average
Accuracy	58.8%
Precision	57.8%
Recall	59.3%
F1-score	58.4%

Validation scores of all seven candidates (macro averages), best first:

Candidate	Accuracy	Precision	Recall	F1-score
Logistic Regression, C = 4 (selected)	67.9%	65.0%	66.7%	65.6%
Logistic Regression, C = 2	67.1%	64.1%	66.4%	64.9%
Logistic Regression, C = 8	67.2%	64.2%	65.3%	64.6%
Logistic Regression, C = 1	65.7%	62.8%	65.8%	63.5%
Logistic Regression, C = 0.5	64.7%	62.1%	65.4%	62.8%
Complement Naive Bayes, alpha = 0.3	62.5%	59.9%	61.8%	60.4%
Complement Naive Bayes, alpha = 1.0	64.0%	62.5%	58.9%	60.0%

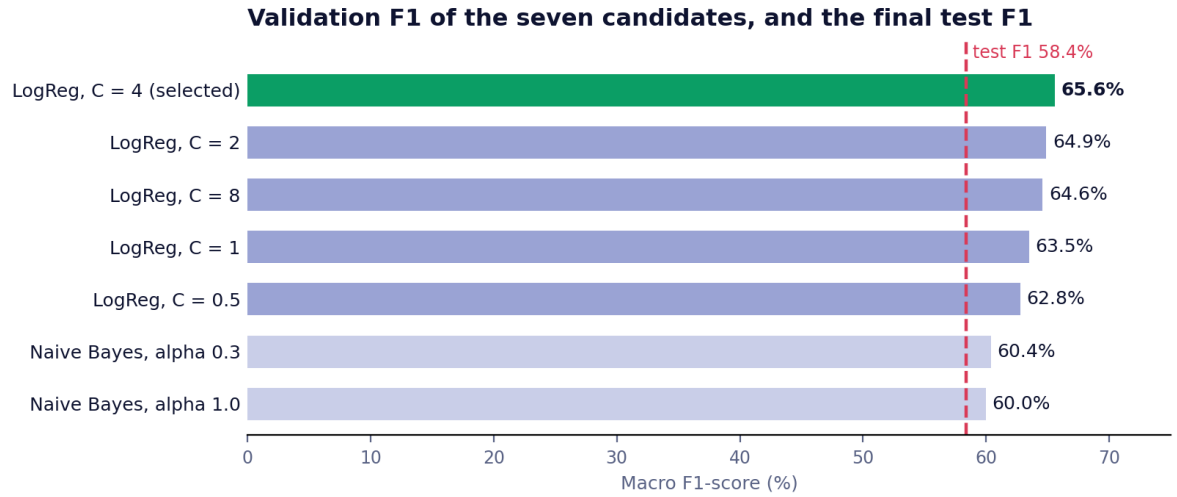


Figure 3. Validation F1 of every candidate, with the final test F1 for comparison

8.3 Discussion

The test scores (accuracy 58.8%, macro F1 58.4%) are lower than the validation scores of the selected model (accuracy 67.9%, macro F1 65.6%). The test split contains different tweets from the training data and may differ in topics and class balance, and the validation split is also the one used to choose the model, so its scores are slightly optimistic. The test figures are the honest estimate of how the model performs on new text.

Logistic Regression beat Complement Naive Bayes on validation F1 by about five points (65.6% against 60.4%), so the simpler baseline was not enough. Per-class precision and recall and the confusion matrix are saved in `ml/artifacts/classification_report.txt` and `ml/artifacts/confusion_matrix.png`. A bag-of-words model also struggles with sarcasm and mixed opinions, which limits how high these scores can go.

9. Backend API design

Method and path	Purpose
`GET /api/health`	Reports whether the service is up and the model is loaded
`GET /api/model-info`	Returns the model name, dataset, split sizes and test metrics
`POST /api/predict`	Accepts `{"text": "..."}` and returns the prediction

A successful prediction returns the sentiment label, the confidence, the probability of each class, the influential keywords and the processing time in milliseconds.

```
{ "sentiment": "Positive", "confidence": 0.87,
  "scores": { "Negative": 0.04, "Neutral": 0.09, "Positive": 0.87 },
  "keywords": [ { "word": "amazing", "sentiment": "Positive" } ],
  "processing_ms": 2.1 }
```

Illustrative response; values vary with the input.

Validation and error handling

The API accepts only JSON containing a string field named `text`, with 3 to 1000 characters after trimming and at least one letter. Request bodies above 32 KB are rejected. Every failure returns the same JSON shape, `{"error": {"code": ..., "message": ...}}`, with an appropriate status code:

Status	Meaning
400	Invalid JSON, wrong type, text too short, too long or without letters
404 and 405	Unknown route or wrong method
413	Request body too large
503	The model file is missing or cannot be loaded
500	Unexpected error; details are logged on the server and never shown to the user

The model is loaded lazily and cached in a thread-safe way, so the first request loads it and later requests reuse it. A security header and configurable CORS headers are added to every response.

10. Frontend design

The interface was designed to look and behave like a real product rather than a classroom demo.

- **Landing page.** A clear headline and explanation, a live analyzer card, a ticker of example comments, a four-step explanation of the pipeline, feature cards and a section showing the model's real metrics.
- **Analyzer.** Large text area with a character counter, three sample texts, a Clear button, a keyboard shortcut (Ctrl+Enter), a loading state and plain-language messages for empty input, network failure and server errors.
- **Result card.** Sentiment label with icon and colour, confidence, three probability bars and highlighted influential words. The page background glow also changes colour to match the latest sentiment.
- **Dashboard.** Totals, counts per class, average confidence, a distribution donut, a seven-day stacked chart, and a history table with search, filter, delete, clear-all and CSV export. Charts are drawn with inline SVG, so no chart library is needed.
- **Authentication page.** Log in and sign-up tabs, Google sign-in, password reset and validation with friendly messages.

Design decisions: one typeface family (Plus Jakarta Sans) with a clear size scale and high-contrast text; glass-style cards over an animated colour field; light and dark themes driven by CSS variables, with the choice remembered; responsive grids that collapse for tablet and phone widths; visible keyboard focus, labelled controls and reduced-motion support; and HTML escaping of all user text before it is displayed, which prevents script injection. If Firebase is not configured, the application falls back to guest mode: analysis still works and the dashboard explains what is missing.

11. Firebase integration and security

Authentication. Users can register and log in with email and password or with Google. Firebase handles password storage, token issuing and password-reset emails.

Cloud Firestore. Each analysis is stored at `users/{uid}/predictions/{id}` with the fields `text`, `sentiment`, `confidence`, `scores` and `createdAt` (server time). The dashboard reads only the signed-in user's collection.

Security rules. The rules in `firebase/firestore.rules` enforce that:

- a user can read, create and delete only documents under their own uid;
- a new document must contain exactly the expected fields, text of 1 to 1000 characters, one of the three sentiment values, a confidence between 0 and 1, a map of scores and a creation time equal to the server time;
- documents cannot be updated after creation;
- every other path in the database is closed.

Credentials. The Firebase web configuration is public by design; it identifies the project and is protected by the rules and by the list of authorised domains. Service-account keys and other private credentials are never used by this project, and `.gitignore` excludes common credential file names as an extra safeguard.

12. Deployment

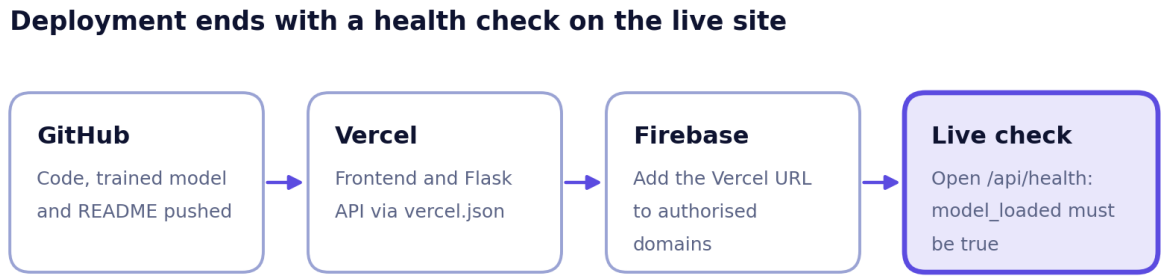


Figure 4. Deployment flow from GitHub to a verified live site

1. The trained model file is committed with the code, because the serverless function must load it at start-up. `scikit-learn` is pinned in `requirements.txt`, and the same version must be used for training so the saved model loads correctly.
2. On Vercel the repository is imported with the framework preset *Other* and no build command. `vercel.json` sets `frontend` as the output directory, includes the `backend` and `models` folders in the Python function and rewrites `/api/*` to the Flask function.
3. The Vercel domain is added in Firebase under Authentication, Authorised domains, so Google sign-in works on the live site.
4. The deployment is verified by opening `/api/health`, which must report that the model is loaded, and then by making a real prediction on the live page.

If the Python function ever exceeds Vercel's size or time limits, the API can be hosted on another platform; the frontend would then set `API_BASE` to that URL and the Vercel frontend would remain the primary live link.

13. Testing

13.1 Automated tests

Seventeen automated tests were written with Python's `unittest` and all pass. They cover text cleaning (lower-casing, URLs and mentions, negations kept, repeated letters, non-string input); Positive, Negative and Neutral predictions through the API; the exact JSON response shape and that class probabilities sum to one; empty text, wrong field name, wrong type, numbers-only text, over-long text and malformed JSON; unknown routes and wrong methods; the model-information endpoint; and a clean 503 response when the model file is missing.

These tests train a very small model from a bundled file (`ml/sample_data/smoke_test.csv`) so that they run without internet access. They prove that the code behaves correctly; they say nothing about model quality, which is judged only by the evaluation in section 8.

13.2 Browser and local tests

The frontend was exercised in a real browser engine (Chromium through Playwright): empty-input error, Positive and Neutral predictions, keyword highlighting, live metrics, no horizontal scrolling at phone width, sign-up validation, history saving, dashboard counts, filtering, searching, deleting, redirects for signed-in users and absence of JavaScript errors. Those runs used a simulated Firebase. The application was also run locally with the trained model, and the home page, the metrics endpoint and the prediction endpoint responded successfully.

13.3 Test plan for the final system

Area	Test	Expected result
Prediction	Positive, Negative and Neutral samples on the live site	Correct label and confidence shown
Input	Empty text, one word, over 1000 characters	Clear message, no crash
Connectivity	Live page calls the API	Result appears without running anything locally
Firebase Auth	Register, log out, log in, Google sign-in, password reset	Each works on the Vercel domain
Firestore	Analyze while signed in, open the dashboard	New row appears; another account cannot see it
Responsive	Desktop, tablet and phone widths	No overflow, readable text
Deployment	Open /api/health	model_loaded is true

14. Challenges and limitations

- **Domain gap.** The training texts are English tweets. Long product reviews, other languages and highly technical text may be classified less accurately.
- **Neutral class.** Neutral is intrinsically the hardest class because it overlaps with both others.
- **Bag-of-words limits.** TF-IDF models do not understand word order beyond bigrams, so sarcasm, irony and mixed opinions can be misread.
- **Serverless constraints.** Cold starts and package-size limits favoured a lightweight model and a small runtime dependency list.
- **Honesty of explanations.** The highlighted words reflect the model's learned weights; they explain the model, not necessarily the true meaning of the text.

15. Future work

- Train on product-review data, or combine datasets, to match the intended use more closely.
- Compare against a transformer model such as DistilBERT for higher accuracy, balancing it against hosting cost and speed.
- Add batch analysis of an uploaded CSV of reviews.
- Support additional languages, including Urdu and Roman Urdu.
- Add monitoring, rate limiting and a feedback button so users can correct wrong predictions for future retraining.

16. Conclusion

Sentra demonstrates the complete path from data to a deployed product: a documented dataset, a reproducible training and evaluation pipeline, a validated and tested Python API, a polished responsive frontend, secure per-user storage with Firebase and a public deployment. Its predictions come from a trained model whose test results are reported as measured (58.8% accuracy, 58.4% macro F1), its security does not depend on hidden credentials, and its limitations are stated openly. The same structure can serve as the foundation for a larger feedback-analysis product.

17. References

1. Barbieri, F., Camacho-Collados, J., Espinosa Anke, L. and Neves, L. (2020). *TweetEval: Unified Benchmark and Comparative Evaluation for Tweet Classification.* Findings of EMNLP 2020.

2. Rosenthal, S., Farra, N. and Nakov, P. (2017). *SemEval-2017 Task 4: Sentiment Analysis in Twitter.* Proceedings of SemEval-2017.
3. Dataset page: https://huggingface.co/datasets/cardiffnlp/tweet_eval
4. Pedregosa, F. et al. (2011). *Scikit-learn: Machine Learning in Python.* Journal of Machine Learning Research, 12.
5. Flask documentation: https://flask.palletsprojects.com
6. Firebase documentation (Authentication, Cloud Firestore, Security Rules): https://firebase.google.com/docs
7. Vercel documentation (Python runtime and configuration): https://vercel.com/docs

Appendix A. Running the project

1. Create a virtual environment and run `pip install -r ml/requirements-train.txt`.
2. Run `python ml/train.py` to download the dataset, train, evaluate and save the model and metrics.
3. Run `python run_local.py` and open http://localhost:5000.
4. Run `python -m unittest discover -s backend/tests -t . -v` to execute the automated tests.
5. Follow `firebase/SETUP.md` to connect Firebase, then the Deployment section of `README.md` to publish on Vercel.